

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Phase-1

Software Requirements Specification

for

Role-Specific AI Voice Assistant Framework for Organizations and Entities

Version 1.0 approved

Team Name: NexGen | Team ID: 43

Team Member	SRN
Varun Kumar H L	PES2UG24AM231
Tejaswi Ganapati Hegde	PES2UG24AM171
Yashas V	PES2UG24AM188
Sri Kanth G	PES2UG24AM161

Date: 01-09-2026

Prepared by: Varun Kumar H L, Tejaswi Ganapati Hegde, Yashas V, Sri Kanth G

Revision History

Name	Date	Reason For Changes	Version
NexGen	01-09-2026	Initial draft of SRS	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for the Role-Specific AI Voice Assistant Framework for Organizations and Entities (Project ID 42), being developed as part of the Software Engineering (UE24CS341A) mini-project (Jackfruit) at PES University. The system is a customizable, voice-enabled virtual assistant framework that can be configured and deployed to serve multiple functional roles within an organization – such as an HR Assistant, an IT Support Assistant, and an Admissions Assistant – from a single underlying platform. Each role assistant is tailored through a configurable role profile that defines its supported intents, conversation flows, and the internal organizational tools or systems it is authorized to interact with.

This document covers Version 1.0 of the proposed system and is intended to guide the requirements, planning, design, implementation, and testing phases of the project as described in the Jackfruit mini-project guidelines.

1.2 Intended Audience and Reading Suggestions

Readers are advised to review Section 2 (Overall Description) first for context, then Section 5 (System Features) for the detailed functional requirements relevant to their role assistant of interest, followed by Section 6 for cross-cutting non-functional requirements.

- **Developers:** use this document to understand the functional requirements, system behavior, interfaces, and operating environment needed to implement the framework.
- **Project Manager:** uses this document to understand scope, priorities, and dependencies for sprint and iteration planning under the Agile methodology mandated for the course.
- **Testers:** use the functional and non-functional requirements in Sections 5 and 6 to derive unit, integration, and system test cases (Jackfruit Phase-3).
- **End Users (Employees):** can refer to this document to understand the voice-assistant interactions available to them for each configured role.
- **Organization Administrators:** use this document to understand role-configuration, integration, and monitoring capabilities.
- **Project Team and Evaluators:** use this document to review scope, analysis models, system features, and quality requirements as part of the Jackfruit evaluation process.

1.3 Product Scope

The Role-Specific AI Voice Assistant Framework is a web-based platform that allows an organization to deploy one AI voice/chat assistant per functional role (for example HR, IT Support, and Admissions) without building a separate assistant from scratch for each department. A single underlying conversational engine is reused across roles; only the intents, sample utterances, workflow mappings, and integration endpoints differ per role profile.

The system enables employees (users) to:

- Register and log in to the system.
- Select the role-specific assistant they wish to interact with (e.g., HR Assistant, IT Support Assistant, Admissions Assistant).
- Interact with the assistant using voice or typed natural-language queries.
- Have their query converted to text, its intent recognized, and a corresponding workflow executed.
- Receive a spoken and/or textual response generated by the assistant.
- Escalate a query to a human agent when the assistant cannot resolve it.
- View their past conversation history.

For organization administrators, the system enables them to:

- Create and configure role profiles (intents, sample utterances, and workflow mappings) for new or existing organizational roles.
- Configure and manage integrations with internal organizational systems (e.g., HRMS, IT ticketing/ITSM, admissions/CRM systems) through configurable API connectors.
- Manage role-based access control for users, administrators, and human agents.
- Monitor assistant usage and view analytics and reports through an administrative dashboard.
- Review escalated conversations and reassign them to available human agents.

The primary objective of the system is to reduce the effort and cost of building and maintaining separate AI assistants for each organizational function by providing a single, configurable, role-aware

voice assistant framework, thereby improving employee self-service, reducing repetitive query load on human staff, and enabling faster resolution of common role-specific requests.

1.4 References

[1] Team, Role-Specific AI Voice Assistant Framework – Use Case and Activity Diagrams, Version 1.0, 2026 (attached in Section 4 and in the project GitHub repository).

[2] Jackfruit Phase-1 Software Requirements Specification Template, Version 1.0, provided by the Department of Computer Science and Engineering, PES University, August 2026.

[3] Jackfruit Mini-Project Guidelines, Subject Code UE24CS341A, Department of Computer Science and Engineering, PES University, 2026.

[4] Team, Role-Specific AI Voice Assistant Framework – Functional and Non-Functional Requirement List, Version 1.0, 2026 (attached in project GitHub repository).

2. Overall Description

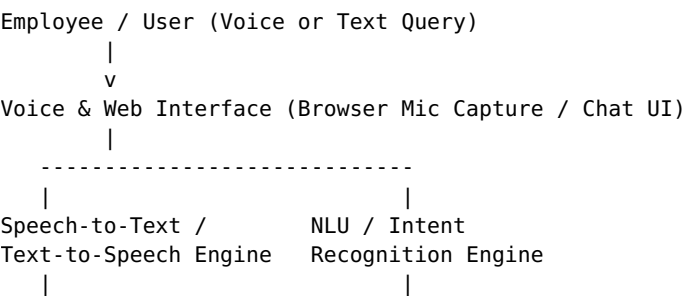
2.1 Product Perspective

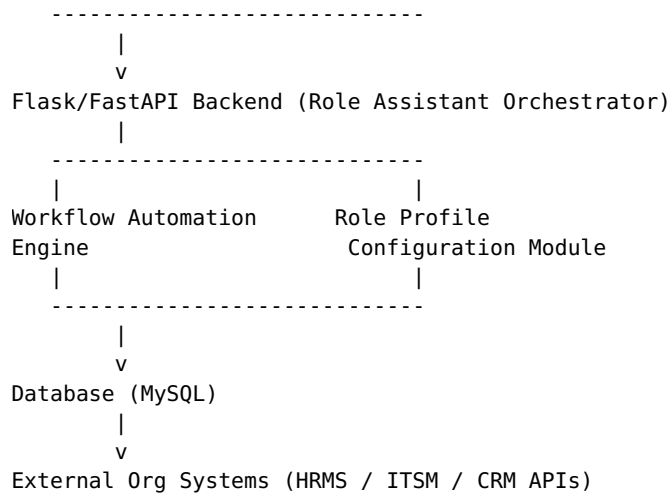
The Role-Specific AI Voice Assistant Framework is a new, self-contained academic prototype developed as part of this project. It is not a replacement for, or extension of, any existing production assistant, but is designed to demonstrate how a single conversational-AI backend can be parameterized to serve multiple organizational roles. The system follows a client-server, layered architecture in which users interact with the assistant through a web-based voice/chat interface; a backend server processes speech, recognizes intent, executes the mapped workflow, and returns a response, while a database persists role profiles, users, conversations, and integration configuration.

The system consists of three primary user roles:

- **User (Employee):** Logs in, selects a role assistant, converses with it by voice or text, receives responses, views conversation history, and escalates unresolved queries.
- **Organization Administrator:** Configures role profiles, manages integrations with internal systems, manages role-based access control, and monitors usage through an administrative dashboard.
- **Human Agent:** Receives and handles conversations escalated by the assistant when it cannot resolve a query with sufficient confidence.

Figure 2.1: High-level architecture of the Role-Specific AI Voice Assistant Framework





The system will be implemented using Python as the primary programming language, with Flask (or FastAPI) as the backend web framework, a speech-to-text/text-to-speech pipeline for voice interaction, and an NLU/intent-recognition component for understanding user queries.

2.2 Product Functions

The Role-Specific AI Voice Assistant Framework provides the following major functions:

User Management - User registration and login. - Role-based access control for users, administrators, and human agents.

Role Assistant Configuration - Creation and configuration of role profiles (e.g., HR, IT Support, Admissions) with associated intents, sample utterances, and workflow mappings. - Selection of a role assistant by the user before starting a conversation.

Conversational Interaction - Voice input capture and speech-to-text conversion. - Text-based query input as an alternative to voice. - Intent recognition and entity extraction from the user query. - Text-to-speech conversion of the generated response.

Workflow Automation - Mapping of recognized intents to organization-specific workflows or API actions. - Execution of the mapped workflow, including calls to configured internal-tool integrations.

Integration Management - Configuration and management of connectors to internal organizational systems (HRMS, ITSM, CRM, etc.).

Escalation Management - Escalation of low-confidence or unresolved queries to a human agent. - Handling of escalated conversations by human agents.

Conversation History and Notifications - Storage and retrieval of conversation history. - Notifications to users regarding escalation and resolution updates.

Administrative Management - Administrative dashboard for monitoring assistant usage. - Reports and analytics on intents handled, escalation rate, and role-wise usage. - Search and filtering of conversations and users.

These functions collectively support the end-to-end lifecycle of a role-specific conversation: role selection, query capture, intent recognition, workflow execution or escalation, response generation,

and logging.

2.3 User Classes and Characteristics

i) User (Employee) Users are employees or members of the organization who interact with a role-specific assistant to get quick answers or complete simple transactional requests (e.g., check leave balance, raise an IT ticket, ask an admissions query). Users are expected to have basic computer/smartphone literacy and require no specialized technical expertise.

ii) Organization Administrator Administrators are authorized personnel responsible for configuring role assistants and managing the framework for their organization. They are expected to have moderate technical familiarity with the organization's internal systems (HRMS, ITSM, CRM) in order to configure integrations.

iii) Human Agent Human agents are support staff who receive conversations escalated by the assistant. They require access only to the escalation queue and the relevant conversation context.

Role-based access control is used to ensure that each user class can access only the functions permitted for its respective role.

2.4 Operating Environment

Client Environment - Desktop, laptop, or smartphone capable of running a modern web browser with microphone access. - Modern web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge. - Stable internet/network connectivity for voice streaming and API calls.

Server Environment - A server capable of hosting the Python runtime, the web framework, and the speech-processing components.

Database Environment - MySQL will be used as the database management system to store user information, role profiles, intents, workflow mappings, integration configuration, conversation history, escalation records, and analytics data.

Application Environment - Development and testing will be performed on standard computer systems supporting Python, Flask, MySQL, a speech-to-text/text-to-speech library or API, and the required web-server components.

2.5 Design and Implementation Constraints

- **Technology Stack Constraint:** The system shall be implemented using Python as the primary programming language, Flask/FastAPI as the web application framework, and MySQL as the database management system.
- **Speech Processing Constraint:** Speech-to-text and text-to-speech functionality shall use an available open-source library or a free-tier cloud speech API, given the academic scope of the project.
- **Time and Resource Constraint:** The project shall be developed and completed within the stipulated academic timeline (Jackfruit Phases 1–3) and the available team resources.
- **Prototype Scope Constraint:** The prototype shall demonstrate a limited number of role assistants (HR Assistant, IT Support Assistant, and Admissions Assistant) rather than an exhaustive set of organizational roles.
- **External Integration Constraint:** Integrations with internal organizational systems (HRMS, ITSM, CRM) shall be simulated using mock services or sandbox/test APIs, since the project shall

not depend on access to a real production organizational system.

- **Data Privacy Constraint:** User personal information and conversation content shall be stored securely and shall not be unnecessarily exposed through publicly accessible interfaces.
- **Browser Compatibility Constraint:** The web application shall support commonly used modern web browsers with microphone-access permission support, including Google Chrome, Mozilla Firefox, and Microsoft Edge.
- **Scalability Constraint:** The system is designed as an academic proof-of-concept and is not intended to support large-scale, multi-tenant, enterprise-wide deployment as part of the current project.

2.6 Assumptions and Dependencies

Assumptions: - Users are assumed to have access to a device with a supported modern web browser, a working microphone (for voice interaction), and a working internet connection. - Users are assumed to provide valid information during registration and login and to maintain the confidentiality of their account credentials. - Organization administrators are assumed to have sufficient knowledge of their organization's processes to correctly configure role profiles, intents, and workflow mappings. - The predefined intents configured for each role are assumed to cover the majority of queries handled by that role assistant; unmatched queries are handled through escalation. - Human agents are assumed to be available, at least during testing and demonstration, to receive escalated conversations.

Dependencies: - The system depends on the availability of the Flask/FastAPI framework and the Python runtime environment for application execution. - The system depends on the availability and proper functioning of the MySQL database for storing and retrieving user, role-profile, conversation, and integration data. - The voice-interaction functionality depends on the availability of a speech-to-text and text-to-speech library or service. - The workflow-execution functionality depends on the availability of the configured internal-system integrations (mocked or sandboxed for this project). - The system depends on a stable network connection for communication between the client, the backend, and the database server.